

CrowdStrike's FalconPy: A Case Study in Modern Python Package Best Practices

Part I: Comprehensive Analysis of the FalconPy SDK

This analysis provides a comprehensive evaluation of the CrowdStrike Falcon SDK for Python, falconpy. The package serves as an exemplary case study in the design, maintenance, and governance of a modern, enterprise-grade open-source software library. The following scorecard offers an executive summary of the detailed findings presented in this report.

Feature/Attribute	Rating	Justification & Key Evidence
Onboarding & Discoverability	Excellent	Consistent, professional presence across GitHub, PyPI, and a dedicated developer portal establishes immediate credibility and simplifies discovery. ¹
Installation Flexibility	Excellent	Caters to a full spectrum of developer workflows, from simple pip installs to sophisticated dependency management with Poetry/pipenv and pre-release channels. ³
API Design & Usability	Excellent	A strategic dual-interface model ("Service" vs. "Uber" classes) brilliantly balances discoverability for new users with scalability and forward-compatibility for power users. ²
Documentation Ecosystem	Good	Reference documentation is exhaustive and systematically generated, ensuring complete API coverage. However, it lacks narrative, task-oriented

		tutorials, relying instead on code examples. ⁵
Security Governance	Excellent	The SECURITY.md policy is a gold standard, providing a transparent, detailed contract with users that includes support matrices and vulnerability disclosure workflows. ²
Dependency Management	Excellent	The project maintains a minimal runtime dependency footprint, drastically reducing the attack surface. Proactive scanning and mitigation of supply chain vulnerabilities are evident. ²
Testing & CI Rigor	Excellent	The CI pipeline is exceptionally rigorous, testing not just against Python versions and operating systems, but also against multiple global production cloud environments. ²
Community & Contribution Model	Good	A welcoming "corporate steward" model provides stability and clear direction, with well-defined contribution guidelines. This prioritizes reliability over community-led agility. ²

Section 1: The Developer Experience (DX) Audit

A developer's journey with a software library begins long before the first line of code is written. It starts with discovery and is shaped by every interaction, from installation to mastering the API. This section audits the end-to-end developer experience offered by falconpy.

1.1 Onboarding and First Impressions: A Multi-Channel Welcome

The initial impression of a software project is critical for adoption, and falconpy excels in this regard by establishing a professional and cohesive presence across multiple developer-centric platforms. The project's entry points—the GitHub repository, the Python Package Index (PyPI) page, and the official CrowdStrike Developer Portal—are synchronized in their messaging and presentation.¹ The GitHub README is well-structured, immediately presenting an "Overview" of the package's purpose and a "Quick Start" guide to demonstrate its value.² Similarly, the PyPI landing page is rich with metadata, including project classifiers, links to documentation and source code, and clear installation instructions, which enhances discoverability and conveys a high degree of professionalism.⁷

This coordinated and polished presence across a dedicated developer portal, which features a portfolio of SDKs for various languages (Go, PowerShell, Python), and meticulously maintained package registry pages is not the result of ad-hoc effort.¹ It is a clear indicator of a formal Developer Relations (DevRel) strategy executed by CrowdStrike. Such a strategy requires dedicated resources, planning, and investment to ensure consistency and quality. For an enterprise evaluating falconpy for adoption, this is a powerful, non-obvious signal of long-term commitment. It suggests the project is not a peripheral side-project but a strategic asset for the company. This institutional backing de-risks adoption by providing strong assurance of continued maintenance, support, and evolution in alignment with the core Falcon platform.

1.2 Installation and Environment Setup: Catering to the Full Spectrum of Developers

The installation process is a developer's first hands-on interaction with a package, and any friction at this stage can deter adoption. falconpy demonstrates a sophisticated understanding of modern Python development workflows by providing a flexible and robust set of installation options. The documentation provides clear guidance for the standard pip installation, ensuring a low barrier to entry for users who need to get started quickly.⁷

Beyond this basic case, the project offers comprehensive instructions for contemporary dependency management tools like Poetry and pipenv.³ This caters to professional software engineers who require deterministic and reproducible builds for production applications. Furthermore, falconpy demonstrates an advanced understanding of developer needs by providing a "bleeding edge" release channel, distributed via TestPyPI, which allows contributors and power users to test the latest features before they are officially released.⁴ This multi-channel release strategy is a hallmark of a mature project. The recommendation to use pipx for installing the related Falcon-Toolkit command-line application further underscores the maintainers' expertise in the Python tooling ecosystem, as pipx is the best practice for installing Python applications in isolated environments.¹¹

This array of installation options can be seen as an expression of user empathy applied to the development lifecycle. The maintainers have clearly considered the different personas who will interact with their package: the data scientist performing a quick analysis, the security

engineer writing an automation script, and the application developer building a long-lived service. By meeting developers in their preferred environment—whether it's a simple local pip install or a Poetry-managed virtual environment—the project minimizes friction, broadens its potential user base, and ultimately drives higher adoption.

1.3 API Design and Usability: Solving the Discoverability vs. Scalability Dilemma

The core architectural decision of the falconpy SDK is its innovative dual-interface approach, a design choice that elegantly addresses the common tension between ease of use and long-term scalability. The SDK offers two distinct methods for interacting with the CrowdStrike API: granular "Service Classes" and a monolithic "Uber Class".²

- **Service Classes:** Each Service Class, such as Hosts or Detects, maps directly to a single API service collection.⁸ This design is highly intuitive and discoverable. A developer tasked with managing hosts can logically infer that they need to from falconpy import Hosts. This one-to-one mapping works exceptionally well with modern IDEs, enabling features like tab-completion and inline documentation, which significantly flattens the learning curve for new users.
- **The Uber Class:** In contrast, the APIHarnessV2 class operates as a single, unified entry point to the entire API surface, which encompasses over 94 distinct service collections.⁵ A developer can instantiate this single object and use its generic command() method to execute any operation by passing the unique "Operation ID" as an argument.¹²

This dual-class architecture is not merely a stylistic preference; it is a sophisticated solution to the challenges of creating a client library for a large and continuously evolving API. The Service Class model optimizes for discoverability and developer ergonomics, making the vast API feel approachable. The Uber Class model optimizes for power, convenience, and, most critically, forward-compatibility. If CrowdStrike releases a new API endpoint, a developer using the Uber Class can interact with it immediately by calling falcon.command("NewOperationID",...) without waiting for a new falconpy release that includes a dedicated Service Class for it.⁸ This prevents the SDK from becoming a bottleneck to accessing the latest platform features.

Both approaches completely abstract away the complexities of OAuth2 authentication, automatically managing token acquisition and renewal in the background, allowing developers to focus solely on their application logic.² The following table deconstructs the practical trade-offs of this design, providing a decision-making framework for developers.

Criterion	Service Class Approach	Uber Class Approach	Recommended Use Case
Discoverability & IDE Support	Excellent. Class and method names are intuitive and easily discovered via autocompletion.	Fair. Requires knowing the specific string-based "Operation ID" for the desired action.	Quick scripts, single-purpose tools, and developers new to the Falcon API.

Code Verbosity (Single Task)	Low. from falconpy <code>import Hosts; falcon = Hosts(...); falcon.query_devices()</code>	Moderate. from falconpy <code>import APIHarnessV2; falcon = APIHarnessV2(...); falcon.command("QueryDevicesByFilter")</code>	N/A
Code Verbosity (Multiple Services)	High. Requires importing and instantiating a separate class for each API service collection.	Low. A single APIHarnessV2 object can access all services.	Complex applications that interact with many different parts of the Falcon platform.
Forward Compatibility (New Endpoints)	Poor. Cannot access new API endpoints until the SDK is explicitly updated and released.	Excellent. Can call any API operation via its ID, even those not yet formally defined in the library.	Long-lived production services where access to the latest API features is critical.
Type Hinting & Static Analysis	Strong. Methods and parameters are explicitly defined, enabling robust static analysis.	Weaker. The generic <code>command()</code> method with <code>**kwargs</code> is less amenable to static type checking.	N/A

1.4 The Documentation Ecosystem: Comprehensive Reference, Sparser Guidance

The falconpy project provides extensive documentation, primarily hosted on a dedicated website, falconpy.io, with additional information available on the GitHub repository's wiki.⁵ The documentation is highly structured and exceptionally comprehensive in its coverage. It features dedicated pages for each of the 94 service collections, which in turn detail every available API operation.⁵ For each operation, the documentation provides the unique Operation ID, the corresponding PEP-8 compliant method name, and exhaustive tables listing every possible keyword argument, its data type, and a description.³

The systematic and uniform nature of this documentation strongly suggests it is automatically generated from the API's underlying OpenAPI (formerly Swagger) specification. This is a best practice for large-scale APIs, as it ensures that the documentation is always accurate and in perfect synchronization with the API's capabilities. It guarantees that every endpoint and every parameter is documented, eliminating the risk of documentation drift.

However, this approach results in a "reference-first" documentation style. It excels at answering the question, "What does this specific parameter do?" but is less effective at answering the more practical question, "How do I accomplish this multi-step goal?" For example, the documentation perfectly describes the `query_devices_by_filter` and

get_device_details operations but does not provide a narrative guide on how to combine them to achieve a common task like "find all Windows servers with a specific vulnerability and retrieve their network interface details." This creates a knowledge gap between the available API "building blocks" and the "blueprints" for common workflows. This explains why the project relies heavily on a separate samples directory and community forums to provide the task-oriented context and narrative examples that the reference documentation inherently lacks.²

1.5 Learning Resources and Community Support: Learning by Example

The primary learning methodology for falconpy is centered on code examples. This "learning by example" approach is consistently applied throughout the project's resources. Every single operation detailed in the official documentation is accompanied by copy-paste-ready code snippets demonstrating its use with both the Service Class and Uber Class syntaxes.³ The GitHub repository reinforces this strategy by maintaining a dedicated /samples directory and actively encouraging the community to submit new examples via pull requests.² Community interaction and support are channeled through well-organized GitHub Discussions, which are categorized into sections like "Q&A" and "Show and Tell," and a reasonably active community on Reddit.⁷ These forums serve as the primary venue for users to ask workflow-related questions that are not covered by the reference documentation. What is notably absent from the project's official resources are narrative, long-form tutorials. Searches for step-by-step guides or tutorials for falconpy were either irrelevant to the package itself or led to community Q&A threads where users were piecing together solutions from examples.¹³ This prioritization of atomic examples over guided tutorials appears to be a deliberate strategic choice. Writing and, more importantly, maintaining high-quality narrative tutorials for a large and rapidly evolving API is an extremely time-consuming endeavor. Providing a comprehensive library of accurate, single-operation examples is a far more scalable and sustainable approach. This strategy implicitly targets a more experienced developer audience, assuming that users have the requisite knowledge to formulate their own solutions by composing the provided snippets. While highly efficient for its target audience, this approach may present a steeper learning curve for developers who are new to APIs, the Python language, or the cybersecurity domain.

Section 2: Security Posture and Risk Assessment

For a library that provides programmatic access to a security platform, its own security posture is of paramount importance. This section assesses falconpy's adherence to modern cybersecurity best practices, from its governance policies to its management of the software supply chain.

2.1 Security Governance and Policy: The Gold Standard

The falconpy repository contains a SECURITY.md file, a standard location for a project's security policy.² However, the content of this file is far from standard; it is exceptionally thorough and sets a benchmark for open-source project security governance.² The policy is not merely a brief instruction to email a security contact. Instead, it functions as a comprehensive and transparent contract with the user community.

The policy explicitly defines the project's support matrix, detailing which versions of Python and which operating systems are actively tested and supported. It outlines a clear, multi-channel process for reporting potential vulnerabilities, including GitHub issues, discussions, and a dedicated email address. Crucially, it sets a service-level objective (SLO) for communication, promising a response to reporters within 48-72 business hours. The document goes on to detail the internal disclosure and mitigation workflow, from confirming the issue to auditing the codebase for similar problems and preparing patches for release.² This level of detail is a direct reflection of the maintainer's (CrowdStrike's) core business in cybersecurity. The policy anticipates the rigorous questions that a corporate risk assessment, legal, or compliance team would ask when evaluating a new third-party dependency. For a Chief Information Security Officer (CISO) or security architect considering the adoption of falconpy, this document is a significant asset. It demonstrates that the project is managed with a level of security rigor that mirrors their own internal standards, building a high degree of trust and making it substantially easier to approve for use within a security-conscious enterprise.

2.2 Dependency Management and Supply Chain Security: Proactive and Minimalist

falconpy exemplifies two fundamental principles of modern software supply chain security: minimizing the attack surface and actively monitoring that surface with automated tooling. The project's runtime dependencies are exceptionally lean, consisting only of the requests library and its own dependency, urllib3.² This is a deliberate and impactful architectural choice. Every third-party dependency introduces a potential vector for vulnerabilities. By keeping the runtime dependency graph to an absolute minimum, the project drastically reduces its exposure to supply chain attacks.

Development dependencies, such as tools for testing and linting, are correctly segregated into a separate requirements-dev.txt file, ensuring that they are not bundled into production deployments.² Within this file lies direct evidence of a proactive security posture: a comment explicitly notes that the ipython package was pinned to a specific version to mitigate a Snyc-identified vulnerability (SNYK-PYTHON-IPYTHON-3318382).² This single line demonstrates that the development team is not passively waiting for vulnerability reports from the community. Instead, they are integrating automated security scanning tools like Snyc into

their development workflow to proactively identify and remediate vulnerabilities in their dependencies. Public checks of the Snyk vulnerability database confirm that recent versions of falconpy have no known direct vulnerabilities.¹⁸ This combination of a minimal attack surface and active, automated monitoring represents a blueprint for secure dependency management.

2.3 Code Quality and Static Analysis: Gated Commits

The falconpy repository contains configuration files for standard Python code quality tools, including .flake8 for style guide enforcement and .pylintrc for static code analysis.² The project's documentation explicitly states that the code is PEP-8 compliant, the official style guide for Python code.²

More significantly, the project's development dependencies include bandit, a static analysis security testing (SAST) tool designed specifically to find common security vulnerabilities in Python code, such as hardcoded credentials or unsafe use of serialization libraries.² The inclusion of these linting and SAST tools in the defined development toolchain, combined with the continuous integration (CI) process that runs on every commit, strongly implies the use of an automated quality gate. It is highly probable that code that fails these automated style, quality, and security checks is automatically blocked from being merged into the main branch. This practice elevates code quality from a mere recommendation to an enforced standard. It creates a system where code quality and basic security hygiene are non-negotiable prerequisites for contribution. This automated enforcement prevents regressions, ensures a consistent and readable codebase, and maintains a high standard of quality across all contributions, whether they originate from an internal CrowdStrike maintainer or an external community member. It is a key characteristic of a mature and highly maintainable software project.

2.4 Licensing and Legal Compliance: The Permissiveness Paradox

The falconpy project is consistently and clearly licensed under "The Unlicense," a legal tool that dedicates the software to the public domain.² The stated intent of this choice is to make the software as free and unencumbered as possible, removing any licensing burden for the user and allowing it to be copied, modified, and distributed for any purpose without restriction.⁵

While the goal of maximizing user freedom is laudable, this choice can, paradoxically, create friction in the environments it is most likely to be used: large enterprises. Corporate legal and compliance departments are highly risk-averse and are trained to work with a set of well-understood, standard open-source licenses like MIT, Apache 2.0, or BSD. These licenses contain explicit grants of rights, including clauses that address patents, which legal teams are specifically looking for. The Unlicense, by dedicating the work to the public domain, lacks

these explicit clauses. This absence of familiar legal language can create ambiguity and uncertainty for a corporate legal team, often triggering a more intensive and time-consuming legal review than a standard MIT license would require. Thus, a license chosen for its ultimate simplicity can inadvertently complicate the adoption process in the most risk-averse organizations.

Section 3: Open-Source Project Health and Best Practices

This section examines the operational maturity of the falconpy project, evaluating its repository structure, contribution model, automation practices, and release management processes as indicators of its long-term health and viability.

3.1 Repository Structure and Maintainability: Modern and Tidy

The structure of the falconpy GitHub repository adheres to modern Python packaging conventions, signaling a high degree of technical currency and maintainability.² The project utilizes a `src/falconpy` source layout, a best practice that prevents a common class of import path problems that can arise during installation and testing.

Furthermore, the repository includes a `pyproject.toml` file. This is the contemporary standard, defined in PEP 518, for configuring Python project metadata and build tooling. Its presence indicates that the project has moved beyond older standards like `setup.py` and `requirements.txt` for core package definition. Adopting modern standards like the `src` layout and `pyproject.toml` is not a trivial undertaking; it often requires a deliberate migration effort. This demonstrates that the project's maintainers are actively keeping the codebase and its tooling aligned with the evolving best practices of the wider Python ecosystem. For a potential adopter, this is a strong indicator of ongoing, high-quality maintenance. It builds confidence that the project is not in a state of passive support but is being actively tended to and modernized, ensuring it will remain compatible and well-supported for the foreseeable future.

3.2 Contribution and Governance Model: The Corporate Steward

The falconpy project operates under a "corporate steward" governance model. The repository contains clear `CONTRIBUTING.md` and `CODE_OF_CONDUCT.md` files that welcome community contributions and set guidelines for positive interaction.² The project's messaging encourages community involvement in various forms, from submitting bug reports and feature ideas to sharing sample code and improving documentation.²

However, the project is explicitly "maintained by CrowdStrike," which clearly defines the governance structure.² While the community is invited to drive the project forward, the ultimate authority for its direction, roadmap, and release schedule rests with the corporate

maintainers. This is a common and highly effective model for corporate-backed open source. It provides a high degree of stability, predictability, and reliability. Users can be confident that the SDK will be promptly updated to support new features of the CrowdStrike Falcon platform. The trade-off is a potential reduction in agility compared to a purely community-led project, as contributions are most likely to be prioritized and accepted if they align with CrowdStrike's strategic roadmap.

3.3 Automation, Testing, and Continuous Integration: Enterprise-Grade Rigor

The project's continuous integration (CI) and testing processes are exceptionally rigorous and represent an enterprise-grade commitment to quality. The presence of a .github directory confirms the use of GitHub Actions for automation², and the project's security policy provides a detailed view into the scope of this automation.²

On every commit and pull request, the codebase is subjected to a comprehensive suite of unit tests run against an extensive test matrix. This matrix covers:

- **Multiple Python Versions:** From 3.7 through 3.13, ensuring backward compatibility and readiness for future interpreters.
- **Multiple Operating Systems:** macOS, Windows, and Ubuntu, covering the primary platforms used by developers.
- **Multiple CrowdStrike Cloud Regions:** US-1, US-2, EU-1, and US-GOV-1.

The inclusion of different cloud regions in the CI test matrix is a profound indicator of the project's commitment to reliability. This goes far beyond standard unit testing, which typically only verifies the logical correctness of the code in an isolated environment. By testing against their entire global production infrastructure, the maintainers are validating the SDK's real-world performance and compatibility. This requires a significant investment in CI infrastructure, as it involves provisioning and authenticating to test environments in each geographical region. This process can detect and mitigate a class of subtle, environment-specific bugs—such as those related to network latency or regional API endpoint variations—that most open-source projects would never discover until a user in that region reported a problem. This is enterprise-grade quality assurance applied directly to an open-source project.

3.4 Versioning, Releases, and Changelog: Disciplined but Potentially Undersold

The falconpy project demonstrates excellent release discipline. The repository contains a CHANGELOG.md file, providing users with a clear history of changes, bug fixes, and new features for each version.² The project has a long and consistent release history on GitHub, with 96 releases documented at the time of this analysis, indicating a steady pace of development and maintenance.² Furthermore, the project offers an automated pre-release channel via TestPyPI for every merged pull request containing a specific trigger string,

allowing for rapid testing of new changes.⁷

Despite this evident maturity, a notable discrepancy exists in the project's packaging metadata. Several versions of the package listed on PyPI carry the development status classifier "4 - Beta".⁷ This classifier sends a signal to the community that the software may be unstable or incomplete, which stands in stark contrast to the project's robust testing, long release history, and enterprise-grade maintenance. This messaging mismatch is likely an oversight in the packaging configuration, but it could have tangible negative consequences. Many automated dependency analysis tools and risk-averse enterprise policies are configured to flag or block the use of packages that are not explicitly marked as "5 - Production/Stable." This metadata discrepancy, while a small detail, could be creating a false perception of risk and actively limiting the project's adoption by causing it to be filtered out by the very users it is designed to serve.

Part II: The Universal Checklist for High-Quality Git Packages

The preceding analysis of falconpy provides a masterclass in building and maintaining a high-quality software package. The principles and practices observed can be generalized into a universal checklist for any team seeking to create, evaluate, or improve their own open-source or internal software packages. This checklist is divided into three key areas: Developer Experience, Security, and Project Management.

Section 4: The Developer Experience (DX) Checklist

This checklist focuses on the end-to-end journey of a developer using the package.

Onboarding & Documentation

- [] **Clear README:** Does the README.md on the main repository page contain a concise value proposition, a one-line installation command, and a minimal "Quick Start" code example?
- [] **Dedicated Documentation Site:** Is there a dedicated, versioned documentation website beyond the README?
- [] **Reference vs. Narrative:** Does the documentation provide both comprehensive API reference material (e.g., auto-generated from source code or specifications) and task-oriented narrative guides or tutorials that show how to solve real-world problems?
- [] **Discoverability:** Is the package published to the standard public repository for its language (e.g., PyPI, npm, Maven Central) with rich metadata and keywords?

Installation & Setup

- ☐ **Standard Tooling:** Does the installation process use the standard, idiomatic package manager for the language and ecosystem?
- ☐ **Environment Support:** Are there clear instructions for different development environments (e.g., local installation, virtual environments like venv/Poetry, containerized environments like Docker)?
- ☐ **Pre-release Channel:** Is there a documented process for users to install and test pre-release, "beta," or "bleeding edge" versions of the package?

API Design & Usability

- ☐ **Boilerplate Abstraction:** Does the library abstract away necessary but repetitive boilerplate code, such as authentication, network retries, or complex configuration?
- ☐ **Intuitive Naming:** Are classes, functions, and parameters named in a way that is clear, predictable, and easily discoverable with IDE autocompletion?
- ☐ **Scalable Design:** Does the API design accommodate both simple, single-use cases and complex, multi-faceted applications? Is there a clear path for users to grow from simple to advanced usage patterns?

Learning & Support

- ☐ **Code Examples:** Are there copy-paste-ready code examples for all primary functions and features, ideally embedded directly within the reference documentation?
- ☐ **Sample Application Library:** Is there a repository or directory of complete, real-world sample applications that demonstrate how to integrate the package into a larger project?
- ☐ **Community Venue:** Has a clear and welcoming venue been established for community questions and support (e.g., GitHub Discussions, Discord, Slack)?

Section 5: The Security and Compliance Checklist

This checklist focuses on the security posture and legal hygiene of the package.

Governance & Policy

- ☐ **SECURITY.md:** Does the repository contain a SECURITY.md file?

- [] **Vulnerability Reporting Process:** Does the security policy define a clear and private channel for reporting security vulnerabilities?
- [] **Support Policy:** Does the policy set expectations for which versions of the package and its dependencies (e.g., language runtimes) are actively supported with security patches?

Supply Chain Security

- [] **Minimal Dependencies:** Are the package's runtime dependencies kept to an absolute minimum?
- [] **Dependency Separation:** Are development-only dependencies (e.g., testing frameworks, linters) clearly separated from runtime dependencies?
- [] **Automated Scanning:** Is an automated dependency scanning tool (e.g., Snyk, GitHub Dependabot) integrated into the project to detect vulnerabilities in third-party code?
- [] **Pinned Dependencies:** Are dependencies pinned to specific, known-good versions in a lock file (e.g., poetry.lock, package-lock.json) to ensure reproducible and verifiable builds?

Secure Coding Practices

- [] **Static Analysis (SAST):** Is a Static Analysis Security Testing (SAST) tool integrated into the CI pipeline to automatically scan for common coding vulnerabilities?
- [] **Automated Linting:** Is code style and quality enforced with automated linters as part of the CI process to prevent the merging of non-compliant code?

Licensing & Compliance

- [] **OSI-Approved License:** Has a standard, OSI-approved license (e.g., MIT, Apache-2.0) been chosen for the project?
- [] **LICENSE File:** Is the full text of the license included in a LICENSE file at the root of the repository?
- [] **Dependency License Compliance:** Has an audit been performed to ensure that the project's use of its dependencies is compliant with their respective licenses?

Section 6: The Project Management and Maintenance Checklist

This checklist focuses on the operational maturity and long-term health of the project.

Repository Hygiene

- [] **Standard Layout:** Does the repository follow the standard directory layout and conventions for its language and ecosystem?
- [] **Modern Tooling:** Does the project use modern, community-accepted standards for configuration and build tooling (e.g., pyproject.toml in Python)?
- [] **Comprehensive.gitignore:** Is the .gitignore file comprehensive, preventing common temporary files, build artifacts, and secrets from being committed?

Contribution & Governance

- [] **CONTRIBUTING.md:** Does the repository have a CONTRIBUTING.md file that explains how to set up a development environment, run tests, and submit changes?
- [] **CODE_OF_CONDUCT.md:** Is there a CODE_OF_CONDUCT.md file to foster a positive and inclusive community environment?
- [] **Clear Governance Model:** Is the project's governance model (e.g., community-led, Benevolent Dictator for Life, corporate steward) clear to all contributors and users?

CI/CD & Automation

- [] **Automated Testing on Commits:** Is every pull request and commit to the main branch automatically built and tested by a CI server?
- [] **Comprehensive Test Matrix:** Does the test suite run against a matrix of all supported language versions, operating systems, and critical external service environments?
- [] **Automated Releases:** Is the process of building, packaging, and publishing releases to the package registry automated to prevent manual error?

Release Management

- [] **Public Changelog:** Is a user-friendly CHANGELOG.md maintained to document changes in each release?
- [] **Consistent Versioning:** Does the project follow a consistent and well-understood versioning scheme, such as Semantic Versioning ()?
- [] **Accurate Package Metadata:** Is the metadata in the public package registry (e.g., development status, supported Python versions, project links) accurate and kept up-to-date with every release?

Works cited

1. CrowdStrike Developer Center, accessed October 6, 2025, <https://developer.crowdstrike.com/>
2. CrowdStrike/falconpy: The CrowdStrike Falcon SDK for ... - GitHub, accessed October 6, 2025, <https://github.com/CrowdStrike/falconpy>
3. Hosts - FalconPy, accessed October 6, 2025, <https://www.falconpy.io/Service-Collections/Hosts.html>
4. Installation, Upgrades & Removal - FalconPy, accessed October 6, 2025, <https://www.falconpy.io/Usage/Installation-Upgrades-and-Removal.html>
5. FalconPy, accessed October 6, 2025, <https://www.falconpy.io/Home.html>
6. Downloads - FalconPy, accessed October 6, 2025, <https://www.falconpy.io/Service-Collections/Downloads.html>
7. crowdstrike-falconpy - PyPI, accessed October 6, 2025, <https://pypi.org/project/crowdstrike-falconpy/0.8.2/>
8. crowdstrike-falconpy - PyPI, accessed October 6, 2025, <https://pypi.org/project/crowdstrike-falconpy/0.9.0/>
9. CrowdStrike SDKs, accessed October 6, 2025, <https://developer.crowdstrike.com/docs/sdks/>
10. Projects - OpenSource @ CrowdStrike, accessed October 6, 2025, <https://opensource.crowdstrike.com/projects>
11. CrowdStrike/Falcon-Toolkit: Unleash the power of the Falcon Platform at the CLI - GitHub, accessed October 6, 2025, <https://github.com/CrowdStrike/Falcon-Toolkit>
12. API Operations Overview - FalconPy, accessed October 6, 2025, <https://www.falconpy.io/Operations/Operations-Overview.html>
13. Getting started with falconpy : r/crowdstrike - Reddit, accessed October 6, 2025, https://www.reddit.com/r/crowdstrike/comments/1eq98wk/getting_started_with_falconpy/
14. Sample Uploads - FalconPy, accessed October 6, 2025, <https://www.falconpy.io/Service-Collections/Sample-Uploads.html>
15. crowdstrike-falconpy · PyPI, accessed October 6, 2025, <https://pypi.org/project/crowdstrike-falconpy/0.5.1/>
16. Tutorial (WSGI) — Falcon 4.1.0 documentation, accessed October 6, 2025, <https://falcon.readthedocs.io/en/stable/user/tutorial.html>
17. Build a Rest API in Python based on falcon - YouTube, accessed October 6, 2025, <https://www.youtube.com/watch?v=W6JvboT8Uo8>
18. crowdstrike-falconpy-dev 1.3.0.dev7 vulnerabilities | Snyk, accessed October 6, 2025, <https://security.snyk.io/package/pip/crowdstrike-falconpy-dev/1.3.0.dev7>